DSA Lab Sheet – 06: Queue

Subject: Data Structure I (BTCS209P) | Language: C++ | Semester: III | Session: 2026–2027

1. Implement Queue Using Array

Aim: Perform Enqueue, Dequeue, Peek and Display.

```
#include <iostream>
using namespace std;
#define MAX 5

int q[MAX], front = -1, rear = -1;

void enqueue(int x) {
    if (rear == MAX-1) cout << "Overflow
";
    else {
        if (front == -1) front = 0;
        q[++rear] = x;
    }
}
void dequeue() {
    if (front == -1) cout << "Underflow
";
    else {
        cout << q[front++] << " deleted
";
        if (front > rear) front = rear = -1;
    }
}
void peek() {
    if (front == -1) cout << "Empty
";
    else cout << q[front] << endl;
}
void display() {
    for (int i = front; front != -1 && i <= rear; i++)
        cout << q[i] << " ";
    cout << endl;
}
int main() {
    enqueue(10); enqueue(20); enqueue(30);
    display(); peek(); dequeue(); display();
}
```

Time: Enqueue/Dequeue/Peek O(1), Display O(n) | Space: O(n)

2. Enqueue — Insert at Rear

```
#include <iostream>
using namespace std;

int main() {
    int q[5], rear = -1, x;
    cout << "Enter value: ";
    cin >> x;
    q[++rear] = x;
    cout << "Inserted at Rear: " << q[rear];
}
```

Time: O(1) | Space: O(n)

3. Dequeue — Delete from Front

```
#include <iostream>
using namespace std;

int main() {
    int q[5] = {10, 20, 30}, front = 0;
    cout << "Deleted: " << q[front++];
}
```

Output: Deleted: 10 | Time: O(1) | Space: O(n)

4. Peek — Display Front Element

```
#include <iostream>
using namespace std;

int main() {
    int q[] = {10, 20, 30}, front = 0;
    cout << "Front = " << q[front];
}
```

Output: Front = 10 | Time: O(1) | Space: O(n)

5. Display Queue from Front to Rear

```
#include <iostream>
using namespace std;

int main() {
    int q[] = {10, 20, 30};
    for (int i = 0; i < 3; i++)
        cout << q[i] << " ";
}
```

Output: 10 20 30 | Time: O(n) | Space: O(n)

6. Demonstrate Queue Overflow

```
#include <iostream>
using namespace std;

int main() {
    int q[3], rear = -1;
    for (int i = 1; i <= 4; i++) {
        if (rear == 2) cout << "Overflow
";
        else q[++rear] = i;
    }
}
```

Output: Overflow | Time: O(1) per insertion | Space: O(n)

7. Demonstrate Queue Underflow

```
#include <iostream>
using namespace std;

int main() {
    int front = -1, rear = -1;
    if (front == -1)
        cout << "Queue Underflow";
}
```

Output: Queue Underflow | Time: O(1) | Space: O(1)

8. Menu-Driven Queue

```
#include <iostream>
using namespace std;
#define MAX 5

int q[MAX], f = -1, r = -1;

void enqueue(int x) {
    if (r == MAX-1) cout << "Overflow
";
    else {
        if (f == -1) f = 0;
        q[++r] = x;
    }
}
void dequeue() {
    if (f == -1) cout << "Underflow
";
    else {
        cout << q[f++] << " deleted
";
```

```
        if (f > r) f = r = -1;
    }
}
void display() {
    for (int i = f; f != -1 && i <= r; i++)
        cout << q[i] << " ";
    cout << endl;
}
int main() {
    enqueue(10); enqueue(20); enqueue(30);
    display();
    dequeue();
    display();
}
```

Time: Enqueue/Dequeue O(1), Display O(n) | Space: O(n)

9. Count Queue Elements

```
#include <iostream>
using namespace std;

int main() {
    int q[] = {10, 20, 30, 40}, count = 4;
    cout << "Elements = " << count;
}
```

Output: Elements = 4 | Time: O(1) | Space: O(n)

10. Search an Element in Queue

```
#include <iostream>
using namespace std;

int main() {
    int q[] = {10, 20, 30, 40}, x = 30;
    for (int i = 0; i < 4; i++)
        if (q[i] == x) {
            cout << "Found at index " << i;
            return 0;
        }
    cout << "Not Found";
}
```

Output: Found at index 2 | Time: O(n) | Space: O(n)

11. Find Maximum and Minimum

```
#include <iostream>
using namespace std;

int main() {
    int q[] = {10, 5, 30, 20}, mx = q[0], mn = q[0];
    for (int i = 1; i < 4; i++) {
        if (q[i] > mx) mx = q[i];
        if (q[i] < mn) mn = q[i];
    }
    cout << "Maximum = " << mx << "
Minimum = " << mn;
}
```

Output: Maximum = 30 \ Minimum = 5 | Time: O(n) | Space: O(n)

12. Reverse Queue Elements

Note: Uses STL stack as an auxiliary structure.

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    int q[] = {10, 20, 30};
    stack<int> s;
    for (int x : q) s.push(x);
    cout << "Reversed: ";
    while (!s.empty()) {
```

```
        cout << s.top() << " ";
        s.pop();
    }
}
```

Output: Reversed: 30 20 10 | Time: O(n) | Space: O(n)

13. Circular Queue Using Array

```
#include <iostream>
using namespace std;
#define MAX 5

int q[MAX], f = -1, r = -1;

void enqueue(int x) {
    if ((r+1)%MAX == f) cout << "Full
";
    else {
        if (f == -1) f = 0;
        r = (r+1)%MAX;
        q[r] = x;
    }
}
void dequeue() {
    if (f == -1) cout << "Empty
";
    else {
        cout << q[f++] << " deleted
";
        if (f == r+1 || f == MAX) f = 0;
        if (f == (r+1)%MAX) f = r = -1;
    }
}
int main() {
    enqueue(10); enqueue(20); enqueue(30);
    dequeue();
    enqueue(40);
    cout << "Circular Queue implemented";
}
```

Time: Enqueue/Dequeue O(1) | Space: O(n)

14. Queue Using Linked List

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

Node *front = NULL, *rear = NULL;

void enqueue(int x) {
    Node* n = new Node{x, NULL};
    if (!rear) front = rear = n;
    else rear->next = n, rear = n;
}
void dequeue() {
    if (!front) cout << "Underflow";
    else {
        Node* t = front;
        cout << t->data << " deleted";
        front = front->next;
        if (!front) rear = NULL;
        delete t;
    }
}
int main() {
    enqueue(10); enqueue(20);
    dequeue();
}
```

Output: 10 deleted | Time: Enqueue/Dequeue O(1) | Space: O(n)

15. Linear Queue vs Circular Queue

```
#include <iostream>
using namespace std;

int main() {
    cout << "Linear Queue:
";
    cout << "Fixed array; freed spaces may be wasted.
";
    cout << "Circular Queue:
";
    cout << "Reuses freed spaces efficiently.
";
    cout << "Both: Enqueue O(1), Dequeue O(1).";
}
```

Operation	Time Complexity	Space Complexity
Enqueue	O(1)	O(n)
Dequeue	O(1)	O(n)
Peek	O(1)	O(n)
Display	O(n)	O(n)
Search	O(n)	O(n)
Count	O(1)	O(n)
Maximum/Minimum	O(n)	O(n)
Reverse	O(n)	O(n)